

LAPORAN PRAKTIKUM
STRUKTUR DATA ADT

Disusun Oleh:

Dinda Amelia

(2511531020)

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG 2026

KATA PENGANTAR

Segala puji bagi Allah SWT yang telah memberikan kesempatan serta kemudahan kepada penulis untuk dapat menyelesaikan Laporan Praktikum pada mata kuliah Struktur Data, sehingga Laporan Praktikum ini dapat dikumpulkan dengan tepat waktu. Atas rahmat dan karunianya Laporan Praktikum dapat terselesaikan dengan baik. Sholawat serta salam juga penulis sampaikan kepada baginda tercinta yaitu Nabi Muhammad SAW, yang kita nantikan syafa'atnya di akhirat nanti. Laporan Praktikum ini bertujuan untuk menambah wawasan para pembaca untuk lebih memperdalam ilmu yang ada pada makalah ini.

Laporan pratikum ini disusun sebagai bentuk penanggung jawaban atas pelaksanaan kegiatan pratikum mata kuliah algoritma pemograman yang membahas tentang ADT Java Eclipse. Dalam praktikum ini, penulis tidak hanya belajar tentang Penyusunan laporan ini bertujuan untuk memberikan pemahaman mengenai cara membuat program lebih rapi, mudah dipahami, fleksibel, dan aman.

Dalam penyusunan Laporan Praktikum ini, penulis mengalami banyak kesulitan dan penulis menyadari bahwa Laporan Praktikum ini masih jauh dari kesempurnaan. Untuk itu, penulis sangat mengharapkan kritik dan saran yang membangun demi kesempurnaan pada Laporan Praktikum ini.

Padang, 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I	1
PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	2
1.3 Manfaat Praktikum	2
BAB II.....	3
PEMBAHASAN.....	3
2.1 Dasar Teori.....	3
2.2 Langkah kerja.....	4
2.2.1 Buka eclipse.....	4
2.2.2 Membuat Java Project	4
2.2.3 Class 1 “Jam_2511531020”.....	5
2.2.4 Class 2 “JamDriver_2511531020”	10
2.2.5 Class 3 “JamDriver2_2511531020”	14
2.3 Tugas Praktikum.....	18
2.3.1 class Mobil_2511531020.....	18
2.3.2 class DriverMobil_2511531020.....	19
BAB III.....	22
KESIMPULAN	22
DAFTAR PUSTAKA.....	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam perkembangan dunia pemrograman, kebutuhan akan pengelolaan data yang efisien dan terstruktur menjadi hal yang sangat penting. Seiring dengan semakin kompleksnya sistem perangkat lunak, programmer dituntut untuk mampu membuat program yang tidak hanya berjalan dengan baik, tetapi juga mudah dipahami, dikembangkan, dan dipelihara. Salah satu konsep yang berperan penting dalam hal ini adalah Abstract Data Type (ADT).

Abstract Data Type (ADT) merupakan suatu konsep dalam ilmu komputer yang mendefinisikan tipe data berdasarkan perilaku atau operasi yang dapat dilakukan terhadap data tersebut, tanpa memperhatikan bagaimana implementasi internalnya. Dengan kata lain, ADT menekankan pada “apa yang dilakukan” daripada “bagaimana cara melakukannya”. Hal ini memungkinkan programmer untuk menggunakan suatu struktur data tanpa harus memahami detail implementasinya secara mendalam.

Konsep ADT sangat erat kaitannya dengan struktur data seperti stack, queue, list, tree, dan graph yang sering digunakan dalam berbagai aplikasi, mulai dari sistem sederhana hingga sistem yang kompleks. Oleh karena itu, pemahaman mengenai ADT menjadi dasar yang penting bagi mahasiswa maupun praktisi di bidang teknologi informasi.

Berdasarkan hal tersebut, maka pembahasan mengenai Abstract Data Type (ADT) menjadi sangat penting untuk dipelajari guna mendukung kemampuan dalam merancang dan mengembangkan perangkat lunak yang efisien, terstruktur, dan berkualitas.

1.2 Tujuan

1. Mempelajari konsep waktu (jam, menit, detik) dalam bentuk Abstract Data Type (ADT).
2. memastikan data waktu yang dimasukkan valid melalui fungsi validasi (isValid).
3. menyediakan akses data yang aman menggunakan metode selektor (getter).
4. Untuk memungkinkan perubahan data secara terkontrol melalui metode mutator (setter).
5. mengimplementasikan operasi konversi waktu ke dalam satuan detik dan sebaliknya.

1.3 Manfaat Praktikum

1. Mahasiswa mampu mengolah data waktu dalam program secara terstruktur dan rapi.
2. Mahasiswa mampu Mencegah kesalahn input waktu dengan adanya validasi (misalnya jam tidak lebih dari 23).
3. Meningkatkan keamanan data karena atribut tidak dapat diakses langsung (encapsulation).
4. Memudahkan penggunaan ulang (reusability) pada program lain yang membutuhkan pengolahan waktu.
5. Membantu dalam perhitungan waktu, seperti konversi ke detik atau manipulasi waktu lainnya.

BAB II

PEMBAHASAN

2.1 Dasar Teori

Abstract Data Type (ADT) merupakan konsep dalam ilmu komputer yang digunakan untuk mendefinisikan suatu tipe data berdasarkan perilaku atau operasi yang dapat dilakukan terhadap data tersebut, tanpa memperhatikan bagaimana data tersebut diimplementasikan secara internal. ADT berfokus pada “apa yang dapat dilakukan” oleh suatu data, bukan “bagaimana cara melakukannya”.

Dalam penerapannya, ADT terdiri dari dua komponen utama, yaitu data (atribut) dan operasi (method). Data merupakan nilai atau informasi yang disimpan, sedangkan operasi adalah fungsi yang digunakan untuk memanipulasi atau mengakses data tersebut. Konsep ini sangat penting dalam pemrograman berorientasi objek karena berkaitan erat dengan prinsip enkapsulasi (encapsulation), yaitu menyembunyikan detail implementasi dan hanya menampilkan bagian yang diperlukan.

Salah satu contoh implementasi ADT adalah representasi waktu dalam bentuk jam, menit, dan detik. Dalam ADT tersebut, data disimpan dalam tiga variabel utama, yaitu jam (hh), menit (mm), dan detik (ss). Untuk menjaga keakuratan data, diperlukan proses validasi agar nilai jam berada pada rentang 0–23, serta menit dan detik berada pada rentang 0–59. Hal ini penting agar data yang diolah tetap konsisten dan sesuai dengan aturan waktu yang berlaku.

Selain itu, ADT juga menyediakan metode selektor (getter) untuk mengambil nilai data dan metode mutator (setter) untuk mengubah nilai data. Dengan adanya metode ini, akses terhadap data menjadi lebih terkontrol dan aman. ADT juga dapat dilengkapi dengan fungsi tambahan, seperti konversi waktu ke dalam satuan detik atau sebaliknya, yang berguna dalam berbagai proses perhitungan.

2.2 Langkah kerja

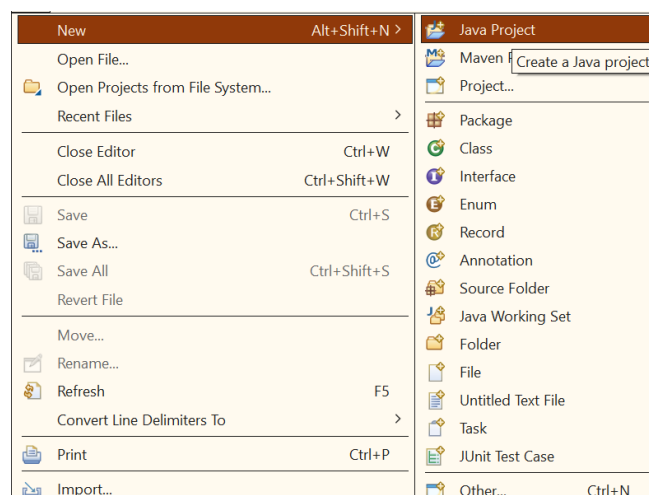
2.2.1 Buka eclipse



Gambar 2.1 Buka eclipse jalankan

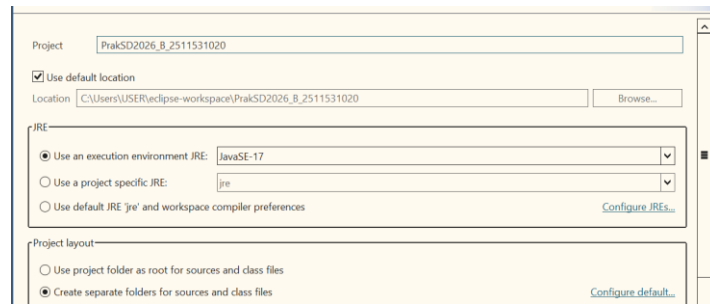
2.2.2 Membuat Java Project

1. Klik file
2. Klik java project



Gambar 2.2 Java Project

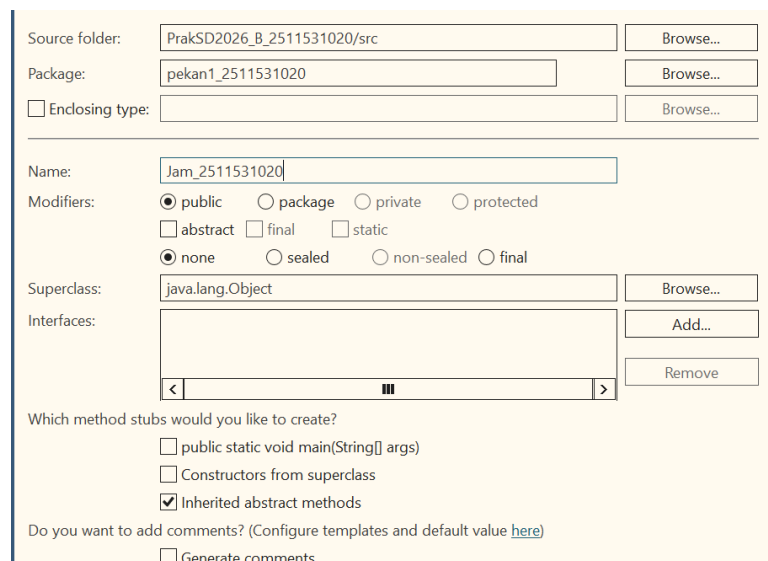
3. Buat nama project



Gambar 2.3 Nama Project

2.2.3 Class 1 “Jam_2511531020”

1. Membuat package pekan1_2511531020 dan class jam_2511531020



Gambar 2.4 Membuat Nama package dan class

2. Klik Finish

3. Kode Program

```
1 package pekan1_2511531020;
2
3 public class Jam_2511531020 {
4     private int hh; // 0..23
5     private int mm; // 0..59
6     private int ss; // 0..59
7
8     // ---Validator---
9     public static boolean isValid(int h, int m, int s) {
10         return (0 <= h && h <= 23) && (0 <= m && m <= 59) && (0 <= s && s <= 59);
11     }
12     // ---Kontruktor/Creator---
13     public Jam_2511531020(int h, int m, int s) {
14         this.hh = h; this.mm = m; this.ss = s;
15     }
16
17     // --- Selektor ---
18     public int getHour() {return hh;}
19     public int getMinute() { return mm;}
20     public int getSecond() { return ss;}
21
22     // --- Mutator ---
23     public void setHour(int h) {this.hh = h;}
24     public void setMinute(int m) { this.mm = m;}
25     public void setSecond(int s) { this.ss = s;}
26
27     //---konversi---
28     public int toSeconds() { return hh * 3600 + mm * 60 + ss;}
29     public static Jam_2511531020 fromSeconds(int total) {
30         if (total < 0) throw new IllegalArgumentException("detik negatif");
31         total = total % (24 * 3600); // wrap 24 jam
32         int h = total / 3600; total = total % 3600;
33         int m = total / 60; int s = total % 60;
34         return new Jam_2511531020(h, m, s);
35     }
36
37     //--- Relasional ---
38     public int compareTo(Jam_2511531020 other) {return Integer.compare(this.toSeconds(), other.toSeconds());}
39     public boolean equals(Object o) {
40         if (!(o instanceof Jam_2511531020)) return false;
41         return hh == j.hh && mm == j.mm && this.ss == j.ss;
42     }
43
44     public int hashCode() {return java.util.Objects.hash(hh, mm, ss);}
45
46     //--- Aritmatika ---
47     public Jam_2511531020 plus(Jam_2511531020 other) {return fromSeconds(this.toSeconds() + other.toSeconds());}
48     public Jam_2511531020 minus(Jam_2511531020 other) { return fromSeconds(Math.floorMod(this.toSeconds() - other.toSeconds(), 24 * 3600));}
49     public Jam_2511531020 nextSecond() { return fromSeconds(this.toSeconds() + 1);}
50     public Jam_2511531020 nextNSeconds(int n) { return fromSeconds(this.toSeconds() + Math.max(0, n));}
51     public Jam_2511531020 prevSecond() { return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 * 3600));}
52     public Jam_2511531020 prevNSeconds(int n) {return fromSeconds(Math.floorMod(this.toSeconds() - Math.max(0, n), 24 * 3600));}
53
54     //--- Durasi (detik) --- bisa negatif jika this > other? sesuai spesifikasi durasi (JAw, JAkh)
55     public static int durasiDetik(Jam_2511531020 jAw, Jam_2511531020 jAkh) { return jAkh.toSeconds() - jAw.toSeconds();}
56     //Overriding ke String
57     public String toString() { return String.format("%02d:%02d:%02d", hh, mm, ss);}
58 }
59
60
```

Gambar 2.5 Class 1 “Jam_2511531020”

4. Analisis Langkah

a. Struktur Kelas

Kelas Jam_2511531020 merepresentasikan sebuah objek waktu dengan atribut:

- hh → jam (0–23)
- mm → menit (0–59)
- ss → detik (0–59)

Kelas ini memiliki fungsi validator, konstruktor, selektor (getter), mutator (setter), konversi, relasional, aritmatika, durasi, dan representasi string.

b. Validator

Java:

```
public static boolean isValid(int h, int m, int s) {  
    return (0 <= h && h <= 23) && (0 <= m && m <= 59) &&  
(0 <= s && s <= 59);  
}
```

- Memastikan nilai jam, menit, dan detik berada dalam rentang valid.
- Berguna untuk mencegah input yang tidak sesuai (misalnya jam = 25).

c. Konstruktor

Java:

```
public Jam_2511531020(int h, int m, int s) {  
    this.hh = h; this.mm = m; this.ss = s;  
}
```

- Membuat objek jam dengan nilai awal tertentu.
- Tidak ada validasi langsung di konstruktor, sehingga bisa saja menerima input invalid jika tidak dicek dengan isValid.

d. Selektor (Getter) & Mutator (Setter)

- Getter: getHour(), getMinute(), getSecond()
- Setter: setHour(), setMinute(), setSecond()

Fungsi ini memungkinkan akses dan modifikasi nilai jam, menit, dan detik.

e. Konversi

Java:

```
public int toSeconds() { return hh * 3600 + mm * 60 + ss; }
public static Jam_2511531020 fromSeconds(int total) {
    if (total < 0) throw new IllegalArgumentException("detik
negatif");
    total = total % (24 * 3600);
    int h = total / 3600; total = total % 3600;
    int m = total / 60; int s = total % 60;
    return new Jam_2511531020(h, m, s);
}
```

- toSeconds() → mengubah jam ke total detik.
- fromSeconds() → mengubah total detik kembali ke format jam:menit:detik, dengan pembatasan 24 jam.

f. Relasional

Java:

```
public int compareTo(Jam_2511531020 other) {return
Integer.compare(this.toSeconds(), other.toSeconds());}
public boolean equals(Object o) {
    if (!(o instanceof Jam_2511531020 j)) return false;
    return hh == j.hh && mm == j.mm && this.ss == j.ss;
}
public int hashCode() {return java.util.Objects.hash(hh, mm,
ss);}
```

- compareTo() → membandingkan dua objek jam berdasarkan total detik.
- equals() → mengecek kesamaan jam, menit, dan detik.
- hashCode() → mendukung penggunaan objek dalam struktur data seperti HashMap.

g. Aritmatika

Java:

```
public Jam_2511531020 plus(Jam_2511531020 other)
{
    return fromSeconds(this.toSeconds() +
        other.toSeconds());
}

public Jam_2511531020 minus(Jam_2511531020 other) {
    return fromSeconds(Math.floorMod(this.toSeconds() -
        other.toSeconds(), 24 * 3600));
}

public Jam_2511531020 nextSecond() {
    return fromSeconds(this.toSeconds() + 1);
}

public Jam_2511531020 nextNSeconds(int n) {
    return fromSeconds(this.toSeconds() + Math.max(0, n));
}

public Jam_2511531020 prevSecond() {
    return fromSeconds(Math.floorMod(this.toSeconds() - 1, 24 *
        3600));
}

public Jam_2511531020 prevNSeconds(int n) {
    return fromSeconds(Math.floorMod(this.toSeconds() -
        Math.max(0, n), 24 * 3600));
}
```

- Mendukung operasi penambahan/pengurangan waktu.
- plus() dan minus() bekerja dengan konversi ke detik.
- nextSecond() dan prevSecond() untuk maju/mundur satu detik.
- nextNSeconds() dan prevNSeconds() untuk maju/mundur sejumlah detik.

h. Durasi

Java:

```
public static int durasiDetik(Jam_2511531020 jAw,
    Jam_2511531020 jAkh) {
```

```

return jAkh.toSeconds() - jAw.toSeconds();
}

```

- Menghitung selisih waktu dalam detik antara dua objek jam.
- Bisa bernilai negatif jika jam akhir lebih kecil dari jam awal.

i. Representasi String

```

public String toString() {return
String.format("%02d:%02d:%02d", hh, mm, ss);}

```

- Menghasilkan format waktu HH:MM:SS dengan dua digit.
- Contoh: 07:05:09.

2.2.4 Class 2 “JamDriver_2511531020”

1. Membuat package pekan1_2511531020 dan class jamDriver_2511531020

Source folder:

Package:

☐ Enclosing type:

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass:

Interfaces:

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Gambar 2.6 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```

1 package pekan1_2511531020;
2
3 public class JamDriver_2511531020 {
4     public static void main(String[] args) {
5
6         Jam_2511531020 a = new Jam_2511531020(23, 59, 50);
7         Jam_2511531020 b = new Jam_2511531020(0, 0, 15);
8
9         System.out.println("a          = " + a);
10        System.out.println("b          = " + b);
11        System.out.println("a + b      = " + a.plus(b));
12        System.out.println("next 20 detik = " + a.nextNSeconds(20));
13        System.out.println("durasi(a,b) = " + Jam_2511531020.durasiDetik(a, b));
14        System.out.println("a.compareTo(b) = " + a.compareTo(b));
15    }
16 }

```

Gambar 2.7 Class 2 “JamDriver_2511531020”

4. Analisis Langkah

a. Tujuan Program

- Menguji kelas Jam_2511531020 dengan membuat objek jam.
- Melakukan operasi penjumlahan waktu, perhitungan durasi, serta perbandingan antar objek jam.
- Menampilkan hasil operasi ke konsol.

b. Struktur Program

- Package: pekan1_2511531020 Menunjukkan bahwa program ini berada dalam paket tertentu.
- Class: JamDriver_2511531020 Berfungsi sebagai *driver class* untuk menguji kelas Jam_2511531020.
- Method utama: public static void main(String[] args) Titik masuk eksekusi program.

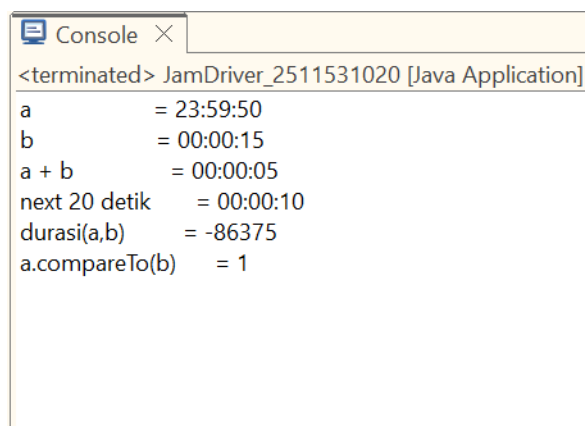
c. Proses Eksekusi

- Jam_2511531020 a = new Jam_2511531020(23, 59, 50);
- Jam_2511531020 b = new Jam_2511531020(0, 0, 15);
- Objek a merepresentasikan jam 23:59:50.
- Objek b merepresentasikan jam 00:00:15.

- d. Menampilkan nilai objek
- `System.out.println("a = " + a);`
`System.out.println("b = " + b);`
 - Output bergantung pada implementasi `toString()` di kelas `Jam_2511531020`.
- e. Operasi penjumlahan waktu
- `System.out.println("a + b = " + a.plus(b));`
 - Menggunakan method `plus()` untuk menambahkan waktu a dengan b.
- f. Operasi penambahan detik
- `System.out.println("next 20 detik = " + a.nextNSeconds(20));`
 - Menghitung jam setelah ditambah 20 detik dari objek a.
- g. Menghitung durasi antar jam
- `System.out.println("durasi(a,b) = " + Jam_2511531020.durasiDetik(a, b));`
 - Method `durasiDetik()` (static) menghitung selisih waktu antara a dan b dalam satuan detik.
- h. Perbandingan antar jam
- `System.out.println("a.compareTo(b) = " + a.compareTo(b));`
 - Method `compareTo()` membandingkan apakah a lebih besar, lebih kecil, atau sama dengan b.
- i. Analisis Konsep OOP
- Encapsulation: Objek jam dibuat dengan konstruktor, menyembunyikan detail implementasi.
 - Method Overloading/Utility: Adanya method `plus`, `nextNSeconds`, dan `durasiDetik` menunjukkan fungsi tambahan untuk manipulasi waktu.

- Polymorphism: Pemanggilan `System.out.println("a = " + a)` memanfaatkan `toString()` untuk menampilkan representasi objek.
- Interface Comparable: Method `compareTo()` mengindikasikan bahwa kelas `Jam_2511531020` mungkin mengimplementasikan Comparable.

5. Output



```
<terminated> JamDriver_2511531020 [Java Application]
a          = 23:59:50
b          = 00:00:15
a + b      = 00:00:05
next 20 detik = 00:00:10
durasi(a,b) = -86375
a.compareTo(b) = 1
```

Gambar 2.8 Output

2.2.5 Class 3 “JamDriver2_2511531020”

1. Membuat package pekan1_2511531020 dan class jamDriver2_2511531020

Source folder: Browse...

Package: Browse...

☐ Enclosing type: Browse...

Name:

Modifiers: ☒ public ☐ package ☐ private ☐ protected
☐ abstract ☐ final ☐ static
☒ none ☐ sealed ☐ non-sealed ☐ final

Superclass: Browse...

Interfaces: Add... Remove

Which method stubs would you like to create?
☐ public static void main(String[] args)
☐ Constructors from superclass
☒ Inherited abstract methods

Do you want to add comments? (Configure templates and default value [here](#))
☐ Generate comments

Gambar 2.9 Membuat Nama package dan class

2. Klik Finish
3. Kode Program

```
1 package pekan1_2511531020;
2
3 import java.util.Scanner;
4
5 public class JamDriver2_2511531020 {
6
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         System.out.println("=== Program Driver Objek Jam ===");
10
11         // 1. Input Jam Pertama
12         System.out.println("\n--- Input Jam 1 ---");
13         Jam_2511531020 j1 = buatJamDariInput(input);
14
15         // 2. Input Jam Kedua
16         System.out.println("\n--- Input Jam 2 ---");
17         Jam_2511531020 j2 = buatJamDariInput(input);
18
19         // 3. Menampilkan Data
20         System.out.println("\n--- Hasil Operasi ---");
21         System.out.println("Jam 1 (String)   : " + j1);
22         System.out.println("Jam 2 (String)   : " + j2);
23         System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
24         System.out.println("Jam 2 dalam detik : " + j2.toSeconds());
25
26         // 4. Operasi Relasional
27         int perbandingan = j1.compareTo(j2);
28         if (perbandingan > 0) {
29             System.out.println("Status           : Jam 1 lebih lambat (setelah) Jam 2");
30         } else if (perbandingan < 0) {
31             System.out.println("Status           : Jam 1 lebih awal (sebelum) Jam 2");
32         } else {
33             System.out.println("Status           : Jam 1 dan Jam 2 sama");
34         }
35     }
36 }
```

```

34 }
35
36 // 5. Operasi Aritmatika
37 System.out.println("Durasi (j1 ke j2) : " + Jam_2511531020.durasiDetik(j1, j2) + " detik");
38 Jam_2511531020.jNext = j1.nextSecond();
39 System.out.println("Next Second (j1) : " + jNext);
40 Jam_2511531020.jPrev = j1.prevSecond();
41 System.out.println("Prev Second (j1) : " + jPrev);
42 Jam_2511531020.jPlus = j1.plus(j2);
43 System.out.println("j1 + j2 : " + jPlus);
44 input.close();
45 System.out.println("\nProgram Selesai.");
46 }
47
48 // Method bantu input + validasi
49 private static Jam_2511531020 buatJamDariInput(Scanner sc) {
50     int h, m, s;
51     while (true) {
52         System.out.print("Masukkan jam (0-23) : ");
53         h = sc.nextInt();
54         System.out.print("Masukkan menit (0-59) : ");
55         m = sc.nextInt();
56         System.out.print("Masukkan detik (0-59) : ");
57         s = sc.nextInt();
58         if (Jam_2511531020.isValid(h, m, s)) {
59             return new Jam_2511531020(h, m, s);
60         } else {
61             System.out.println("[Error] Input tidak valid! Coba lagi.\n");
62         }
63     }
64 }

```

Gambar 2.10 Class 3 “JamDriver2_2511531020”

4. Analisis Langkah

a. Tujuan Program

Program ini dibuat untuk:

- Menguji kelas Jam_2511531020 dengan input dari pengguna.
- Melakukan operasi relasional (perbandingan waktu).
- Melakukan operasi aritmatika (durasi, next/prev second, penjumlahan jam).
- Menampilkan hasil operasi ke konsol dengan format yang jelas.

b. Struktur Program

- Package: pekan1_2511531020 Menandakan program berada dalam paket praktikum minggu pertama.
- Class: Driver_2511531020 Berfungsi sebagai *driver class* untuk menguji objek jam.
- Library: java.util.Scanner Digunakan untuk membaca input dari pengguna.
- Method utama: public static void main(String[] args) Titik masuk eksekusi program.
- Method bantu: buatJamDariInput(Scanner sc) Digunakan untuk membuat objek jam dari input pengguna dengan validasi.

c. Proses Eksekusi

Input jam dari pengguna

- Program meminta pengguna memasukkan jam, menit, dan detik.
- Validasi dilakukan dengan `Jam_2511531020.isValid(h, m, s)`.
- Jika valid, objek jam dibuat; jika tidak, pengguna diminta mengulang.

d. Menampilkan data jam

Java:

```
System.out.println("Jam 1 (String)   : " + j1);
System.out.println("Jam 2 (String)   : " + j2);
System.out.println("Jam 1 dalam detik : " + j1.toSeconds());
System.out.println("Jam 2 dalam detik : " + j2.toSeconds());
```

- Memanfaatkan `toString()` untuk representasi jam.
- `toSeconds()` mengubah jam ke total detik.

e. Operasi relasional

Java:

```
int perbandingan = j1.compareTo(j2);
```

- Jika hasil $> 0 \rightarrow$ Jam 1 lebih lambat dari Jam 2.
- Jika hasil $< 0 \rightarrow$ Jam 1 lebih awal dari Jam 2.
- Jika hasil $= 0 \rightarrow$ Kedua jam sama.

f. Operasi aritmatika

Java:

```
System.out.println("Durasi   (j1   ke   j2)   :   "   +
Jam_2511531020.durasiDetik(j1, j2) + " detik");
Jam_2511531020.jNext = j1.nextSecond();
Jam_2511531020.jPrev = j1.prevSecond();
Jam_2511531020.jPlus = j1.plus(j2);
```

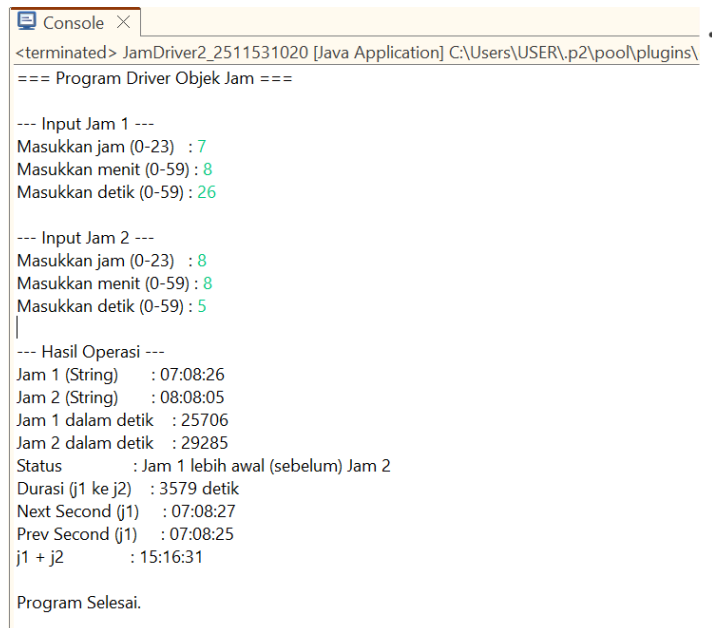
- `durasiDetik(j1, j2)` menghitung selisih waktu dalam detik.
- `nextSecond()` menambah 1 detik pada jam.
- `prevSecond()` mengurangi 1 detik pada jam.
- `plus(j2)` menambahkan jam kedua ke jam pertama.

g. Akhir program

- Scanner ditutup dengan `input.close()`.

- Program menampilkan pesan “Program Selesai.”
- h. Analisis Konsep OOP
- Encapsulation: Input divalidasi melalui method `isValid()`, menjaga agar objek jam selalu konsisten.
 - Polymorphism: Pemanggilan `System.out.println(j1)` memanfaatkan `toString()` untuk menampilkan jam.
 - Inheritance/Interface: Method `compareTo()` menunjukkan kemungkinan implementasi `Comparable`.
 - Static Method: `durasiDetik()` digunakan tanpa membuat objek baru, cukup memanggil langsung dari kelas.

5. Output



```
<terminated> JamDriver2_2511531020 [Java Application] C:\Users\USER\p2\pool\plugins\
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan jam (0-23) : 7
Masukkan menit (0-59) : 8
Masukkan detik (0-59) : 26

--- Input Jam 2 ---
Masukkan jam (0-23) : 8
Masukkan menit (0-59) : 8
Masukkan detik (0-59) : 5
|

--- Hasil Operasi ---
Jam 1 (String) : 07:08:26
Jam 2 (String) : 08:08:05
Jam 1 dalam detik : 25706
Jam 2 dalam detik : 29285
Status : Jam 1 lebih awal (sebelum) Jam 2
Durasi (j1 ke j2) : 3579 detik
Next Second (j1) : 07:08:27
Prev Second (j1) : 07:08:25
j1 + j2 : 15:16:31

Program Selesai.
```

Gambar 2.11 Output

2.3 Tugas Praktikum

2.3.1 class Mobil_2511531020

1. Kode Program

```
1 package pekan1_2511531020;
2
3 public class Mobil_2511531020 {
4     // Atribut
5     private String nama;
6     private int tahun;
7     private int cc;
8     private double harga;
9     private String merk;
10
11     // Constructor
12     public Mobil_2511531020(String nama, int tahun, int cc, double harga, String merk) {
13         this.nama = nama;
14         this.tahun = tahun;
15         this.cc = cc;
16         this.harga = harga;
17         this.merk = merk;
18     }
19
20     // Getter
21     public String getNama() { return nama; }
22     public int getTahun() { return tahun; }
23     public int getCc() { return cc; }
24     public double getHarga() { return harga; }
25     public String getMerk() { return merk; }
26
27     // Setter
28     public void setNama(String nama) { this.nama = nama; }
29     public void setTahun(int tahun) { this.tahun = tahun; }
30     public void setCc(int cc) { this.cc = cc; }
31     public void setHarga(double harga) { this.harga = harga; }
32     public void setMerk(String merk) { this.merk = merk; }
33
34     // Method tampilkan info
35     public void tampilkanInfo() {
36         System.out.println("Nama: " + nama);
37         System.out.println("Tahun: " + tahun);
38         System.out.println("CC: " + cc);
39         System.out.println("Harga: " + harga);
40         System.out.println("Merk: " + merk);
41     }
42 }
```

Gambar 2.12 class Mobil

2. Analisis Langkah

- a. Langkah 1: Mendefinisikan atribut mobil (nama, tahun, cc, harga, merk) sebagai private.
- b. Langkah 2: Membuat constructor untuk menginisialisasi objek mobil dengan data awal.
- c. Langkah 3: Menyediakan getter (selektor) untuk membaca nilai atribut.

- d. Langkah 4: Menyediakan setter (mutator) untuk mengubah nilai atribut.
- e. Langkah 5: Menyediakan method tampilkanInfo() untuk menampilkan detail mobil ke layar.

2.3.2 class DriverMobil_2511531020

1. Kode Program

```

1  package pekan1_2511531020;
2
3  import java.util.ArrayList;
4
5  public class DriverMobil_2511531020 {
6      private Mobil_2511531020[] daftarMobil;
7      private int jumlahMobil; // penghitung jumlah mobil yang terisi
8
9      public DriverMobil_2511531020(int kapasitas) {
10         daftarMobil = new Mobil_2511531020[kapasitas];
11         jumlahMobil = 0;
12     }
13
14     // Method tambah mobil
15     public void tambahMobil(Mobil_2511531020 m) {
16         if (jumlahMobil < daftarMobil.length) {
17             daftarMobil[jumlahMobil] = m;
18             jumlahMobil++;
19             System.out.println("Mobil berhasil ditambahkan: " + m.getNama());
20         } else {
21             System.out.println("Daftar mobil penuh, tidak bisa menambah lagi.");
22         }
23     }
24
25     // Method hapus mobil berdasarkan nama
26     public void hapusMobil(String nama) {
27         for (int i = 0; i < jumlahMobil; i++) {
28             if (daftarMobil[i].getNama().equalsIgnoreCase(nama)) {
29                 // Geser elemen setelahnya ke kiri
30                 for (int j = i; j < jumlahMobil - 1; j++) {
31                     daftarMobil[j] = daftarMobil[j + 1];
32                 }
33                 daftarMobil[jumlahMobil - 1] = null;
34                 jumlahMobil--;
35                 System.out.println("Mobil berhasil dihapus: " + nama);
36                 return;
37             }
38         }
39         System.out.println("Mobil dengan nama " + nama + " tidak ditemukan.");
40     }
41
42     // Method tampilkan semua mobil
43     public void tampilkanSemuaMobil() {
44         if (jumlahMobil == 0) {
45             System.out.println("Tidak ada mobil dalam daftar.");
46         } else {
47             for (int i = 0; i < jumlahMobil; i++) {
48                 daftarMobil[i].tampilkanInfo();
49                 System.out.println("-----");
50             }
51         }
52     }
53
54     // Main method untuk uji coba
55     public static void main(String[] args) {
56         DriverMobil_2511531020 driver = new DriverMobil_2511531020(10);
57         Mobil_2511531020 m1 = new Mobil_2511531020("Avanza", 2020, 1300, 1500000000, "Toyota");
58         Mobil_2511531020 m2 = new Mobil_2511531020("Civic", 2021, 1500, 3500000000, "Honda");
59         driver.tambahMobil(m1);
60         driver.tambahMobil(m2);
61         System.out.println("\nDaftar Mobil:");
62         driver.tampilkanSemuaMobil();
63         driver.hapusMobil("Avanza");
64         System.out.println("\nDaftar Mobil setelah dihapus:");
65         driver.tampilkanSemuaMobil();
66     }
67 }

```

Gambar 2.13 class DriverMobil

2. Analisis Langkah

Langkah 1: Membuat array `Mobil_2511531020[]` `daftarMobil` dengan kapasitas tertentu (misalnya 10).

Langkah 2: Menyediakan variabel `jumlahMobil` untuk melacak berapa mobil yang sudah tersimpan.

Langkah 3: Method `tambahMobil()`:

- Mengecek apakah array masih ada ruang.
- Jika ada, simpan objek mobil ke indeks berikutnya.
- Tambah nilai `jumlahMobil`.
- Cetak pesan bahwa mobil berhasil ditambahkan.

Langkah 4: Method `hapusMobil()`:

- Cari mobil berdasarkan nama.
- Jika ditemukan, geser elemen setelahnya ke kiri agar array tetap rapat.
- Kurangi nilai `jumlahMobil`.
- Cetak pesan bahwa mobil berhasil dihapus.

Langkah 5: Method `tampilkanSemuaMobil()`:

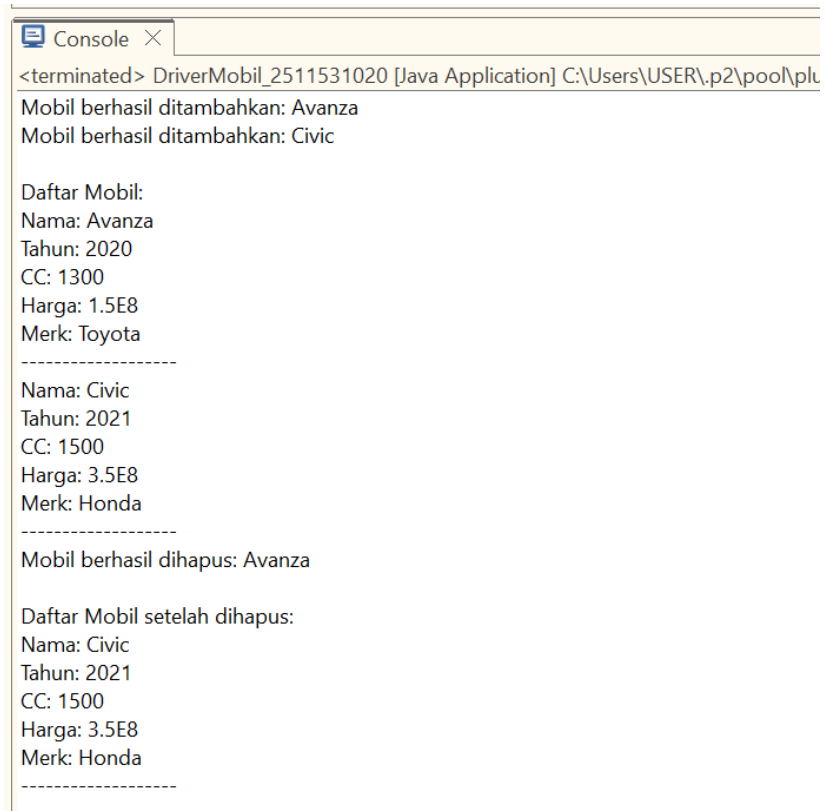
- Jika `jumlahMobil == 0`, tampilkan pesan "Tidak ada mobil".
- Jika ada, loop dari indeks 0 sampai `jumlahMobil-1` dan panggil `tampilkanInfo()` untuk setiap mobil.

Langkah 6: Method `main()`:

- Membuat objek driver dengan kapasitas array.
- Membuat beberapa objek mobil (Avanza, Civic).
- Menambahkan mobil ke daftar.
- Menampilkan semua mobil.
- Menghapus satu mobil.

- Menampilkan daftar mobil setelah penghapusan.

3. Output



```
Console X
<terminated> DriverMobil_2511531020 [Java Application] C:\Users\USER\p2\pool\plu
Mobil berhasil ditambahkan: Avanza
Mobil berhasil ditambahkan: Civic

Daftar Mobil:
Nama: Avanza
Tahun: 2020
CC: 1300
Harga: 1.5E8
Merk: Toyota
-----
Nama: Civic
Tahun: 2021
CC: 1500
Harga: 3.5E8
Merk: Honda
-----
Mobil berhasil dihapus: Avanza

Daftar Mobil setelah dihapus:
Nama: Civic
Tahun: 2021
CC: 1500
Harga: 3.5E8
Merk: Honda
-----
```

Gambar 2.13 Output

BAB III

KESIMPULAN

Dari praktikum ini adalah bahwa program Driver Objek Jam berhasil menunjukkan penerapan konsep dasar *Object-Oriented Programming* (OOP) dalam Java melalui pembuatan, manipulasi, dan perbandingan objek jam. Program mampu menerima input dari pengguna dengan validasi, sehingga memastikan data yang dimasukkan sesuai aturan. Selain itu, berbagai operasi relasional seperti `compareTo()` dan operasi aritmatika seperti `durasiDetik`, `nextSecond`, `prevSecond`, serta plus dapat dijalankan dengan baik untuk memberikan hasil yang informatif. Praktikum ini melatih keterampilan mahasiswa dalam mengintegrasikan input pengguna, memanfaatkan method untuk manipulasi data waktu, serta menampilkan output yang jelas dan terstruktur. Secara keseluruhan, praktikum ini memperkuat pemahaman mengenai bagaimana konsep OOP dapat digunakan untuk memodelkan objek nyata dan melakukan operasi yang relevan dalam kehidupan sehari-hari.

DAFTAR PUSTAKA

- [1] Rachman, D. A. Perkenalan Abstrak Data Type.
- [2] Ikhsan, F. K., Hasan, F. N., Afandi, I. R., Sutjiningtyas, S., Fathurrohman, S., Salma, T. D., ... & Ismanto, E. (2025). *Struktur Data dengan Bahasa Java*. PT Penerbit Qriset Indonesia.
- [3] Amin, M. M., Kom, S., Sutrisman, A., Kom, S., Kom, M., & Stiawan, D. (2019). *Pemrograman Aplikasi Mobile Menggunakan Eclipse ADT: Seri Tutorial Pembuatan Aplikasi Mobile Berbasis Android*. Penerbit Ahatek.
- [4] Emmanuel, W. C., & Monden, A. (2021, November). Human Resource Analysis Based on Used Libraries in Eclipse Projects on GitHub. In *2021 IEEE/ACIS 22nd International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing (SNPD)* (pp. 254-257). IEEE.
- [5] Borges, H., Hora, A., & Valente, M. T. (2016, October). Understanding the factors that impact the popularity of GitHub repositories. In *2016 IEEE international conference on software maintenance and evolution (ICSME)* (pp. 334-344). IEEE.